

Devoir surveillé

Durée 2h
(Corrections)

Nom :
Prénom :
Num. inscription :

Aucun document, outil de calcul ou de communication autorisé.

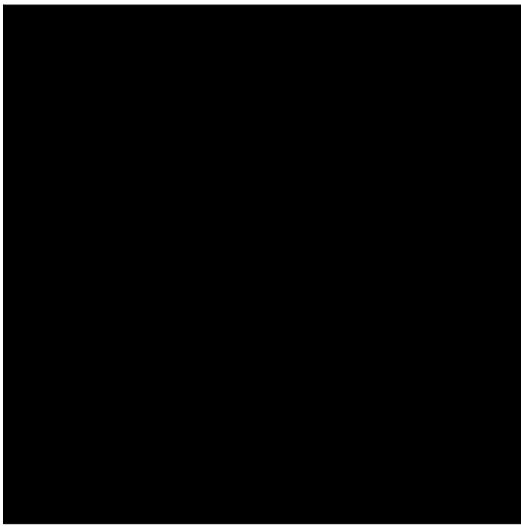
Exercice 1 (QCM - 5 pts)

Vous obtiendrez 0,5 point aux questions si vous cochez exactement toutes les bonnes réponses. Une mauvaise réponse cochée entraîne des points négatifs.

- La taille occupée sur le disque par un fichier...
 - ☒ ...dépend de la taille des blocs
 - ☐ ...est indépendante de la taille des blocs
 - ☒ ...peut être supérieure à sa taille réelle
 - ☒ ...est différente suivant le système de fichiers
- L'écriture d'une structure dans un fichier en utilisant les fonctions de bas-niveau...
 - ☒ ...conserve l'alignement des structures
 - ☐ ...ne conserve pas l'alignement des structures
 - ☒ ...est indépendante de l'interprétation des données
 - ☒ ...risque de ne pas être lisible sur un autre ordinateur
- L'appel à `fork`...
 - ☐ ...produit l'envoi d'un signal au processus père
 - ☐ ...produit l'envoi d'un signal au processus fils
 - ☒ ...duplique les données, mêmes celles allouées dynamiquement
 - ☐ ...duplique les données, sauf celles allouées dynamiquement
- En utilisant le tableau fourni en annexe (page suivante), quelle est la taille de la structure suivante ?

```
typedef struct {  
    long a;  
    char b;  
    char c[2];  
    int d;  
    short e;  
} structure_t;
```

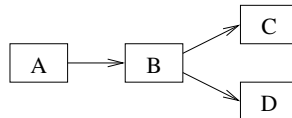
- ☐ 13o ☒ 16o ☐ 24o ☐ Une autre réponse
- En considérant la question 4, combien d'octets de bourrage sont ajoutés par le compilateur ?
 - ☐ 0 ☐ 1 ☒ 3 ☐ Une autre réponse
 - En considérant la question 4, quelle est la taille minimale que l'on peut obtenir en réarrangeant les champs de la structure (sans modifier les types) ?
 - ☐ 13o ☒ 16o ☐ 24o ☐ Une autre réponse



7. Concernant les tubes anonymes :

- ☒ L'écriture n'est pas forcément atomique
- ☐ Par défaut, la lecture n'est pas bloquante
- ☐ Par défaut, l'écriture n'est pas bloquante
- ☒ Les descripteurs de fichier sont transmis lors de la duplication du processus avec `fork`

8. Combien de sémaphores de *Dijkstra* sont nécessaires au minimum pour résoudre les contraintes du diagramme de précédence suivant (A, B, C et D étant des sections de code de processus quelconques) ?

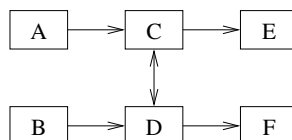


- ☐ 1 ☒ 2 ☐ 3 ☐ 4

9. Soit le diagramme de précédence de la question 8. Combien d'opérations (hors `init`) sont nécessaires au minimum ?

- ☐ 4 ☒ 6 ☐ 8 ☐ 10

10. Combien de sémaphores de *Dijkstra* sont nécessaires au minimum pour résoudre les contraintes du diagramme de précédence suivant (A, B, C, D, E et F étant des sections de code de processus quelconques) ?



- ☐ 1 ☐ 3 ☒ 5 ☐ 7

Exercice 2 (Le voyageur de commerce - 15 pts)

Nous souhaitons réaliser une application distribuée permettant de résoudre le problème du voyageur de commerce. Pour rappel, ce problème consiste à trouver le trajet le plus court entre un ensemble de villes qui passe une et une seule fois par chaque ville. Les distances et les villes sont fixées et connues de toutes les applications.

Nous développerons une solution basée sur le modèle producteur/consommateur. Le producteur crée les tâches (qui correspondent à un début de parcours) et les consommateurs les résolvent (le résultat est le parcours le plus court commençant par le début de parcours spécifié). Le producteur collecte les résultats envoyés par les consommateurs et conserve le meilleur résultat.

Le producteur crée deux fils : le premier, nommé *gestionnaire de connexions*, sera affecté à la gestion des connexions/déconnexions des consommateurs et le second, nommé *gestionnaire de tâches* sera affecté à la génération des tâches et à la récupération des résultats. Les deux fils enverront leurs informations au producteur en utilisant un unique tube anonyme. Le producteur affichera les résultats au fur-et-à-mesure, ainsi que les connexions/déconnexions.

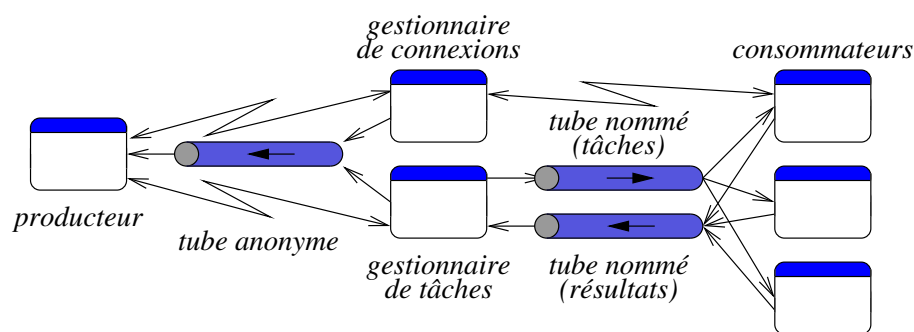
Pour se connecter, les consommateurs échangeront des signaux avec le gestionnaire de connexions. Le nombre maximum de consommateurs est fixé par la constante `MAX_CONSOMMATEURS` : lors de sa demande de connexion, le consommateur doit être averti si le nombre maximum de connexions est atteint. Lorsqu'un consommateur s'arrête (quand il reçoit un signal `SIGINT`), il en informe le gestionnaire de connexions. De même, lorsque le producteur reçoit un signal `SIGINT`, il doit arrêter ses deux fils et les consommateurs éventuels.

Lorsqu'un consommateur est connecté, il lit une tâche dans un tube nommé. Il envoie le résultat dans un second tube nommé. Les tâches et les résultats seront une suite d'entiers (chacun correspondant à une ville) de longueur variable.

Généralités (4 points)

1°) Proposez une modélisation de votre application sous forme de schéma en précisant chaque entité et les éléments systèmes mis en place. (1 point)

Solution : *voici une solution :*



2°) Donnez l'ensemble des échanges entre les différentes entités, en précisant le type d'échange et les données échangées. (1,5 points)

Solution : *Voici les différents échanges :*

- Gestionnaire de connexions → producteur (tube anonyme) :
 - Message connexion : type = `CONNEXION` (`unsigned char`), PID du consommateur (`pid_t`)
 - Message déconnexion : type = `DECONNEXION` (`unsigned char`), PID du consommateur (`pid_t`)
- Gestionnaire de tâches → producteur (tube anonyme) :
 - Tâche calculée : type = `TACHE` (`unsigned char`), parcours (`int[]`)
- Producteur → gestionnaires (signaux) :
 - `SIGINT` : demande d'arrêt
- Gestionnaire de connexions → consommateurs (signaux) :
 - Demande d'arrêt : `SIGINT`
 - Autorisation/refus : `SIGRTMIN` + entier (0 pour connexion autorisée et 1 pour refus)
- Consommateurs → gestionnaire de connexions (signaux) :
 - Demande de connexion : `SIGRTMIN`
- Gestionnaire de tâches → consommateurs (tube nommé) :
 - Tâche : tableau d'entiers (de taille fixe = nombre de villes)
- Consommateurs → gestionnaire de tâches (tube nommé) :
 - Résultat : tableau d'entiers (de taille fixe = nombre de villes)

3°) Sachant que le nom des tubes est fixé dans les constantes `TUBE_TACHES` et `TUBE_RESULTATS` et que les fonctions correspondant aux deux fils du producteur sont `connexion` et `tache`, donnez le code du producteur permettant de créer les deux fils avec le tube anonyme. (1,5 points)

Solution : *Voici une solution :*

```
int tube[2];
pid_t filsC, filsT;
```

```

pipe(tube);
if((filsC = fork()) == 0) {
    connexion();
    exit(EXIT_SUCCESS);
}
if((filsT = fork()) == 0) {
    tache();
    exit(EXIT_SUCCESS);
}

```

Les signaux (6 points)

4°) Nous supposons que le consommateur envoie un signal **SIGRTMIN** au gestionnaire de connexions avec une valeur 1 (on suppose le PID du gestionnaire de connexions connu). Il reçoit à son tour un signal avec un code d'état lui indiquant s'il peut ou non se connecter. Donnez l'extrait de code du consommateur permettant de mettre en place la phase de connexion. **(2 points)**

Solution : *Le plus simple est d'utiliser `sigwaitinfo` qui permet d'attendre un signal, sans avoir à mettre en place un gestionnaire. Voici une solution :*

```

sigset_t signaux, old;

sigfillset(&signaux);
sigprocmask(SIG_BLOCK, &signaux, &old);

union sigval valeur;
valeur.sival_int = 1;
sigqueue(pidConnexion, SIGRTMIN, valeur);

sigemptyset(&signaux);
sigaddset(&signaux, SIGRTMIN);

siginfo_t infos;
sigwaitinfo(&signaux, &infos);

if(infos.si_value.sival_int == 0)
    printf("Connexion_refusée_par_le_gestionnaire... \n");
else
    printf("Connexion_autorisée_par_le_gestionnaire... \n");

```

5°) Comment le gestionnaire de connexions peut-il répondre au consommateur, sachant que le PID du consommateur n'est pas connu? **(1 point)**

Solution : *Lorsqu'un signal est reçu, la structure `siginfo_t` est remplie. Elle contient notamment le PID du processus qui a envoyé le signal.*

6°) Quand l'utilisateur presse **CRTL + C**, le consommateur s'arrête. Il envoie alors un signal **SIGRTMIN** au gestionnaire de connexions avec une valeur 2. Donnez les extraits de code du consommateur permettant de mettre en place ce mécanisme. **(2 points)**

Solution : *Comme le consommateur doit réaliser des actions, il est plus simple d'utiliser un gestionnaire. Pour mettre en place le gestionnaire :*

```

struct sigaction action;

sigfillset(&action.sa_mask);
action.sa_flags = 0;
action.sa_handler = gestionnaire;
sigaction(SIGINT, &action, NULL);

```

Le gestionnaire :

```
int stop = 0;
void gestionnaire(int num_sig) {
    stop = 1;
}
```

Enfin, la boucle principale du consommateur :

```
while(stop == 0) {
    /* Lecture d'une tâche */

    /* Envoi du résultat */
}
union sigval valeur;
valeur.sival_int = 1;
sigqueue(pidConnexion, SIGRTMIN, valeur);
```

7°) Que se passe-t-il du point de vue de l'exécution d'un programme lorsqu'un signal est reçu ? Expliquez ce que l'on doit mettre en place au niveau de la gestion d'erreur. (1 point)

Solution : Lors de la réception d'un signal, s'il n'entraîne pas l'arrêt du programme, l'exécution est interrompue puis reprise après l'exécution du gestionnaire. Dans le cas d'un appel système, il est interrompu et le retour est -1; la variable globale **errno** prend alors la valeur **EINTR** (interruption). Il est donc nécessaire de vérifier si l'erreur générée correspond à une interruption ou non. À noter que si un **exit** est utilisé dans le gestionnaire associé au signal, le traitement est plus simple.

Les tubes (5 points)

8°) Expliquez le fonctionnement général du gestionnaire de tâches (sans donner le code mais en précisant les appels systèmes nécessaires). (1 point)

Solution : Dans un premier temps, le gestionnaire de tâches doit créer les deux tubes (**mkfifo**). Il les ouvre ensuite en écriture (pour le tube des tâches) et en lecture (pour le tube des résultats) (**open**). Il écrit des tâches dans le tube des tâches et lit ensuite les résultats (**read** et **write**). Dès qu'un résultat est lu, il envoie le résultat au producteur dans le tube anonyme (**write**) et écrit une nouvelle tâche dans le tube.

9°) Quels sont les problèmes de synchronisation liés aux tubes ? Proposez une solution pour chacun. (1 point)

Solution : Lorsque le gestionnaire des tâches ouvre le premier tube, il est bloqué, tant qu'un consommateur ne l'ouvre pas à son tour. Pas de problème de ce côté, même si cela signifie que le gestionnaire de tâches ne commencera pas à remplir le tube pendant ce temps-là.

Si la taille des tâches est trop importante, l'écriture n'est pas atomique. Il faut donc s'arranger que la taille ne dépasse pas la taille spécifiée par la constante système **PIPE_BUF**. Idem pour l'écriture des résultats : sinon, les données risquent d'être mélangées entre les différents consommateurs.

10°) En considérant que la taille des tâches est suffisamment petite, en quoi la taille maximale d'un tube peut-elle poser problème pour l'envoi de tâches et la réception des résultats ? Y'a-t-il une possibilité d'obtenir un cas d'inter-blocage ? Comment l'éviter ? (1 point)

Solution : Si le nombre de tâches est trop important, l'écriture sera bloquante. Tant qu'une tâche ne sera pas lue, le gestionnaire de tâches sera bloqué en écriture. Les résultats ne seront pas lus, mais un consommateur pourra débloquent le gestionnaire de tâches en lisant une nouvelle tâche. Par contre, si le tube des résultats est plein lui-aussi, il y aura alors un cas d'inter-blocage.

Une solution est de réaliser une écriture et une lecture non bloquantes. Le gestionnaire des tâches tente d'écrire, de lire puis se met en pause.

11°) Que se passe-t-il lors de la connexion ou la déconnexion de consommateurs du côté du gestionnaire de tâches ? Expliquez les problèmes liés à l'utilisation des tubes nommés et proposez une solution sans ajouter de moyens de communications supplémentaires entre le gestionnaire de tâches et les consommateurs. (2 points)

Solution : *L'écriture dans un tube échoue s'il n'y a pas d'écrivain. Aussi, si un consommateur s'arrête, le gestionnaire de tâches risque de s'interrompre. Une solution est de vérifier le retour de **read** dans le tube des résultats et de **write**, ainsi que de bloquer le signal **SIGPIPE** (synonyme d'interruption du gestionnaire). Dès que l'un des cas est détecté, le gestionnaire des tâches peut fermer les tubes, par exemple, puis les ouvrir à nouveau : il sera bloqué jusqu'à la connexion d'un nouveau consommateur. Un autre problème : des tâches risquent d'être perdues si un consommateur a lu la tâche et s'arrête avant d'envoyer le résultat. Ce cas est plus difficile à résoudre : utilisation d'une alarme (si je n'ai pas reçu le résultat, je renvoie la tâche), envoie des tâches plusieurs fois (identification unique et vérification lors de la réception des résultats). Une autre solution : le consommateur peut envoyer à nouveau la tâche dans le tube lorsqu'il s'arrête.*